

1 Brief

Design a service for scanning and filtering spam/malicious emails. Specially the service needs to support:

- Scanning an email via an API request.
- Providing access to a specified REST API, e.g. for use by front-end interfaces and internal teams.
- Remaining responsive while scanning emails.
- Being available when called, as an important email infrastructure service.

Task You are working for SpamOverflow, a new competitor in the email security space. SpamOverflow uses a microservices based architecture to implement their new malicious email filtering platform. The CEO saw on your resume that you are taking Software Architecture and has assigned you to design and implement a service. This service must be scalable to cope with a large influx of emails.

Requirements Email filtering software can filter email as it arrives or after. SpamOverflow will implement a service that does not impede the flow of traffic (i.e. does not prevent email from arriving). It will receive an API call when the mail server receives an email message. The service then pulls the email from the user's inbox, as fast as it can, to prevent the user from seeing the malicious email or clicking any links.

Commercial email providers send an API request for each email received. For optimal performance this service needs to be able to handle a large number of requests in a short period of time.

Since these emails can be dangerous, the service must be able to report that it is bad or good in a timely manner. Though genuine emails that are incorrectly marked as dangerous should be returned to the user as quickly as possible.

Persistence is an important characteristic of the platform. Customers will want to analyse why emails were flagged after the fact. Upon receiving an email scan request, and after filtering, the system must guarantee that the data has been saved to persistent storage before returning a success response.

2 Outline

Introduction (5 minutes)

Introduction to the brief and resources, including the [API specification](#)¹ and quality scenarios in section 3. Your service will use a tool to scan emails for malicious content, called [SpamHammer](#)².

¹<https://csse6400.uqcloud.net/api/spamoverflow>

²<https://github.com/CSSE6400/SpamHammer>

Planning (10 minutes)

In small groups, discuss the following issues or any others you think are relevant to designing the service.

1. What are the key requirements introduced by the quality scenarios?
2. What strategies can you implement to support these scenarios?
3. What AWS resources would prove helpful?
4. What are the likely bottlenecks?

 Teacher

2.1 What are the key requirements introduced by the quality scenarios?

- Q1 & Q2 are simple and predictable loads for the system to manage.
- Q3 introduces loads that requiring scaling up and down.
- Q4 adds the need for prioritisation, requiring some form of priority queue.
- Q5 implies that some malicious email will be very similar to other malicious messages.
- Q6 forces processing through the SpamHammer processing bottleneck.

 Teacher

2.2 What strategies can you implement to support these scenarios?

Spam Identification The [SpamHammer documentation^a](https://github.com/CSSE6400/SpamHammer), shows the expected runtime and memory usage of checking if an email message is potentially malicious. We can see that the process is CPU intensive but light on memory, taking up to several seconds to process a message. Running this process sequentially will only work for light loads.

+ **Queue malicious email checks to offload processing.**

^a<https://github.com/CSSE6400/SpamHammer>

 Teacher

High Value Customers The scenario only mentions one priority customer, but the `POST /customers/customer_id/emails` endpoint implies there could be multiple priority customers.

- Simple approach is to have two queues, one for priority processing and one for normal processing. Load balancing should be done separately for each queue. Does not allow management of a specific high value customer's priority.
- Separate queues for each high value customer quickly becomes unwieldy. Requires reconfiguring infrastructure as new high value customers are added.
- Could suggest redesigning the customer IDs to allow identification of each high value customer. If their identity is coded simply into the customer ID, a hash table could be created with each bucket being a message queue. A smart load balancer could then manage worker scaling to meet the performance specified in service level agreements.

+ **Two queues for simplicity.**

Teacher

Leaked Directory Some malicious email will be generated in bulk and will be very similar to each other. Once one of these messages are identified as being malicious, it is possible to use pattern matching to look for similar messages. This could be faster than waiting for a response from SpamHammer for every message.

- Need a mechanism to identify many similar malicious emails are being received. Potentially track number malicious emails identified as begin sent to a single domain over a short time period. Try using pattern matching or similarity metrics to check other emails sent to the same domain.

+ **Similarity metrics and observer process.**

Teacher

Generating Reports Creating reports on demand, once there is a large collection of data, will be resource intensive. Maintain tables of the report data that are updated as emails are processed.

+ **Report data tables.**

Teacher

2.3 What AWS resources would prove helpful?

- Lambda for compute
- ECS for compute
- EC2 for compute
- DynamoDB for querying/caching
- RDS for querying
- Redis for caching/queueing
- S3 for storage
- SQS for queueing

Teacher

2.4 What are the likely bottlenecks?

Primarily email checks due to the SpamHammer processing time.

Teacher

2.5 Availability Requirement

Ask the class whether availability will require any additional design considerations, beyond what they have thought about to maintain responsiveness.

Design (20 minutes)

In your group, design an appropriate architecture for SpamOverflow. You need to consider the flow of an API request through your service, use the [API specification](#) and quality scenarios in section 3 to ensure all use cases have been considered.

Presentation (15 minutes)

In the remaining time, each group should present their proposed architecture design. This is an opportunity for discussion amongst the class to point out limitations of the proposed system designs.

3 Quality Scenarios

Q1: Steady Stream Steady receipt of email messages at a rate of M per minute, fairly evenly spread across all customers. Approximately 20% of the messages are malicious.

Q2: Bad 'News' Stream Steady receipt of email messages at a rate of N per minute, fairly evenly spread across all customers. Approximately 80% of the messages are malicious.

Q3: Peaks and Troughs Periods of receipt of a high volume of email messages, followed by periods of low volume.

Q4: High Value Customer The Department of Defence (DoD) has adopted SpamOverflow. They are a high value customer and you must ensure that their requests are handled quickly. All other customers have a service level agreement (SLA) that guarantees a certain level of responsiveness. You cannot ignore their requests to only prioritise the DoD's requests.

Q5: Leaked Directory A bad actor has managed to get the email addresses of all employees of <Large Company>. They have sent a phishing email to all of the users advertising a pay raise with a link to a fake login page. The email is sent to all 10,000 employees at the same time.

Q6: Personalised Attack A bad actor has trained an AI model using social media profiles of targeted victims. They can generate personalised phishing email messages based on personal information. As the messages are personalised, they can be of greatly different lengths and contain different content. These messages can only be identified by SpamHammer. They have sent these phishing messages to 2,000 users at the same time.